

Application Load Balancer with AutoScaling for a PHP WebApp

Description

1. Host a simple PHP web application in the AWS EC2 instance.
2. Configure an Application Load Balancer (ALB) for the instance tier containing the web application.
3. Create an Auto Scaling Group (ASG) with targeted scaling policies for high availability and resource optimization.
4. Attach the Application Load Balancer directly with the Auto Scaling Group target configurations.
5. Verify the operational working of the horizontal auto-scaling policies by manually injecting system load.

System Architecture Flow

End User → Application Load Balancer (ALB) → Target Group → Auto Scaling Group (ASG) → EC2 Instances (PHP App)

Technical Implementation Steps

Step 1: Create an EC2 Launch Template

- Navigate to EC2 Dashboard —► Launch Templates, and click Create Launch Template.
- Provide a descriptive name (e.g., php-app-launch-template).
- Select a standard Amazon Linux 2023 or Amazon Linux 2 AMI.
- Set the Instance Type to t3.micro (or choose an appropriate free-tier compatible instance like t2.micro depending on account region).
- Select or generate an RSA SSH Key Pair for secure instance access.
- Configure a standard Security Group: Allow inbound SSH (Port 22) restricted to 'My IP' and HTTP (Port 80) open to 'Anywhere (0.0.0.0/0)' to accept load balancer traffic.

Step 2: Network & Availability Zone Configuration

- CRITICAL ARCHITECTURAL REQUIREMENT: Do not hardcode or specify a specific Subnet or Availability Zone within the Launch Template parameters.
- This configuration flexibility allows the Auto Scaling Group to dynamically provision instances across all designated subnets and AZs specified at the group tier level.

Step 3: Define Bootstrap Script (User Data)

- Expand the Advanced Details layout tab and scroll down to the User Data input field.
- Insert the script below to automate system updates, install the Apache HTTP web server, configure PHP extensions, and deploy an application script that exposes the underlying instance's private IP identity.

```
#!/aim/bash
# Ensure compatibility with Amazon Linux 2 / 2023
sudo dnf update -y || sudo yum update -y
sudo dnf install -y httpd php || sudo yum install -y httpd php
sudo systemctl start httpd
sudo systemctl enable httpd

# Generate dynamic PHP application script
cat << 'EOF' > /var/www/html/index.php
<!DOCTYPE html>
<html>
<head>
    <title>PHP Core Target App</title>
    <style>body { font-family: Arial, sans-serif; text-align: center; margin-top: 10%;
background: #f4f6f9; }</style>
</head>
<body>
    <h1>Application Cluster Node</h1>
    <p>Serving Request From Private IP: <strong><?php echo $_SERVER['LOCAL_ADDR'];
?></strong></p>
</body>
</html>
EOF
```

Step 4: Provision an Elastic Load Balancing Target Group

- Navigate to EC2 Cluster Dashboard —► Target Groups —► Create Target Group.
- Choose Target Type: Instances.
- Define Protocol: HTTP, Port: 80.
- Configure Health Checks path setting exactly to: /index.php.
- Advance through without manually registering individual static instances since the Auto Scaling Group handles dynamic tracking lifecycle automatically.

Step 5: Configure Application Load Balancer (ALB)

- Navigate to Load Balancers —► Create Load Balancer —► Application Load Balancer.
- Set scheme option to Internet-facing.
- Network Mapping: Choose the default VPC and select at least two distinct public subnets mapped to different Availability Zones for redundant load handling.
- Listeners and Routing: Configure Listener Protocol: HTTP, Port: 80. Explicitly set actions rule to 'Forward to' your newly designated Target Group.

Step 6: Instantiate the Auto Scaling Group (ASG)

- Navigate to Auto Scaling Groups —► Create Auto Scaling Group.
- Step 1: Choose the Launch Template created in Step 1.
- Step 2: Network setup: Bind to your chosen production VPC and select multiple availability zones/subnets corresponding to your infrastructure availability requirements.
- Step 3: Integrate Advanced Options: Select Attach to an existing load balancer, choose your Target Group from the dropdown, and check the box to Enable ELB Health Checks to ensure automatic recovery of unhealthy web servers.

Step 7: Define Group Size Capacity Limits

- Configure capacity structures to safely scale constraints within specific bounds:
- • Desired Capacity: 1 (Baseline count of active instances)
- • Minimum Capacity: 1 (Lower guardrail boundary)
- • Maximum Capacity: 3 (Upper threshold ceiling scale constraint)

Step 8: Implement Dynamic Target Tracking Scaling Policy

- Choose scaling policy type: Target Tracking Scaling Policy.
- Metric Type: Average CPU Utilization.
- Target Value Value: 15% (A highly sensitive trigger tailored for immediate scaling confirmation under testing scenarios).
- Instance Warmup Period: Set to 60 seconds to prevent over-provisioning lag loops during scaling iterations.

Validation and Testing Protocols

Step 9: Initial Deployment Verification

- Copy the public DNS endpoint of the created Application Load Balancer.
- Paste the domain path address into a web browser. Confirm the baseline configuration page successfully displays the private IP identifier of the running EC2 node instance.

Step 10: Inject Simulated System Workload (Scale-Out Test)

- Establish an SSH connection into the single active running EC2 instance.
- Execute multiple parallel null pipe commands to maximize CPU computation pipeline loops and force metric breaches:

```
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &
yes > /dev/null &

# Monitor real-time processor utilization locally:
top
```

Step 11: Monitor Dynamic Auto Scaling Trigger Activities

- Return to the AWS Cloud Management Console dashboard tier.
- Open Auto Scaling Groups —► Select your Target group —► Activity tab.
- Observe operational logs tracking alarm statuses. Monitor the sequential lifecycle triggers as CloudWatch initializes and shifts properties to scale out, launching subsequent new server instances automatically to handle load.

Step 12: Validate Distributed Request Load Balancing Lifecycle

- Once secondary nodes are registered online and marked healthy by the target health system, return to your browser window tab pointing to the ALB DNS endpoint URL.
- Continuously trigger a hard refresh cycle (Ctrl + F5). Observe alternating private IP string signatures being displayed on the screen, proving structural success of the path routing algorithms.

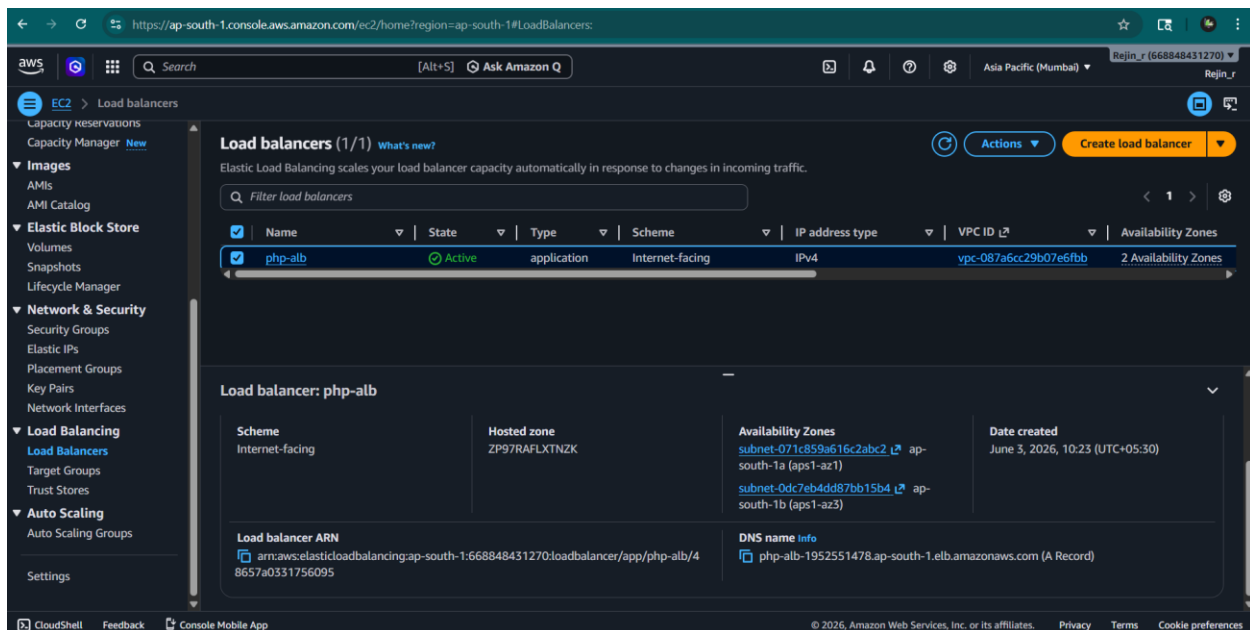
Step 13: Execute System Scale-In Evaluation Cleanup

- In the terminal context panel of the master instance server, execute the kill signal routine to abruptly purge processing demand and allow standard cooling conditions:

```
killall yes
```

- Monitor the ASG Activity tab logs across the next few tracking intervals to confirm automated metric stabilization and safe system node spin down back to baseline structures.

Output Pictures



Auto Scaling group: php-asg				
Successful	Terminating EC2 instance: i-0980963d06de82aab	At 2026-06-03T06:03:45Z a user request update of AutoScalingGroup constraints to min: 1, max: 3, desired: 1 changing the desired capacity from 2 to 1. At 2026-06-03T06:03:53Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 2 to 1. At 2026-06-03T06:03:54Z instance i-0980963d06de82aab was selected for termination.	2026 June 03, 11:33:54 AM +05:30	2026 Ju 03, 11:33:54 AM +05:30
Successful	Launching a new EC2 instance: i-0354d52f9153c952e	At 2026-06-03T05:51:48Z a user request update of AutoScalingGroup constraints to min: 1, max: 5, desired: 2 changing the desired capacity from 1 to 2. At 2026-06-03T05:51:56Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.	2026 June 03, 11:21:58 AM +05:30	2026 Ju 03, 11:21:58 AM +05:30
Successful	Launching a new EC2 instance: i-0980963d06de82aab	At 2026-06-03T05:00:11Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 1. At 2026-06-03T05:03:04Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.	2026 June 03, 10:33:06 AM +05:30	2026 Ju 03, 10:33:06 AM +05:30

Welcome to 172.31.11.179

Welcome to 172.31.39.96

Welcome to 172.31.9.60